

Guidelines for the initial phase

BSc Project: Risk-Bounded, Human-Aware Autonomy in a Search-and-Rescue Testbed

S. Schoonebeek and A. Jamshidnejad

April 2026

1 Purpose of this document

This document provides concrete directions for the first part of the Bachelor project. During this initial phase, you should work together to build the common simulation framework that will serve as the foundation for all later individual sub-projects. At the same time, you should already take the needs of their own sub-projects into account and, where relevant, ensure that these needs are reflected in the design of the shared environment. Some of these design choices may only be useful for a subset of the sub-projects, but this is acceptable as long as the common framework remains coherent and manageable. The goal of the project is both to obtain meaningful results and to deepen your understanding of the underlying topics. For that reason, the framework does not need to be fully realistic or overly complex. At the same time, the final outcomes should be of sufficient quality to support a publishable result, so the framework should be designed carefully, systematically, and with reproducibility in mind.

2 Objectives

The shared baseline should provide a reproducible two-dimensional grid-based [Search-and-rescue \(SaR\)](#) environment with one or more robot agents, simple scenario generation, and clear visualizations of the robot trajectory and map evolution. The environment should expose a small and consistent interface, meaning a clear and minimal set of functions for initializing the simulator, applying actions, retrieving observations, rendering the current state, and computing evaluation metrics.

By the end of this first joint phase, you should have a working and reproducible baseline environment with a parameterized design, so that multiple scenarios can be generated easily by adjusting a limited number of settings. This will be important later for systematic numerical analysis. You should also implement at least one simple reference policy, such as random exploration or greedy scanning, to provide a baseline against which the methods developed in the individual sub-projects can be evaluated. Finally, you should document the state representation, the main modeling assumptions, the key environment parameters, and the principal simplifications that were made.

The shared baseline should already compute a core set of evaluation metrics that are useful across the sub-projects, while leaving room for task-specific extensions later on. At a minimum, these metrics should cover safety-related quantities, such as collisions, illegal moves, or exposure to hazardous regions, as well as effectiveness measures, such as explored area, victims detected, and travel efficiency, and uncertainty-related quantities, such as how much of the map is currently known with confidence and how accurate the maintained beliefs are compared with the underlying ground truth. Depending on the individual sub-project, additional criteria may also become important; for example, tasks involving interaction with a human teammate may require extra metrics related to the human state, inferred intent, cooperation quality, or safety with respect to the human. Basic runtime measurements may also be included from the start, especially because computation time will later matter for the MPC- and RL-based methods.

3 Modeling

The environment can be designed using **matrices that encode properties of the grid**, roughly inspired by a recent grid-based [SaR](#) modeling approach developed in related TU Delft work on model predictive fuzzy control for outdoor search-and-rescue robots. The idea is to separate the world into layers, where each layer stores one interpretable property of the environment. Assume a grid with dimensions

$$\mathcal{G} = \{1, \dots, n_h\} \times \{1, \dots, n_v\}.$$

It is recommended to implement the environment as a collection of matrices with matching dimensions. Use a single discrete time index

$$k = 0, 1, 2, \dots, T.$$

All environment updates, scans, and robot actions should happen once per time step. You should distinguish between:

- **ground truth state** of the environment,
- **perceived state** used by the controller.

This separation is very important. It allows uncertainty-aware control and human-aware reasoning to be studied without excessive mathematical complexity.

3.1 Vulnerability matrix

Define a time-invariant matrix

$$M^{\text{vul}} \in [0, 1]^{n_h \times n_v},$$

where the entry M_{ij}^{vul} encodes a *static property* of the cell (i, j) . Generally, it can represent **cell vulnerability or environmental risk potential**.

A simple interpretation is:

- 0: safe cell,
- 0.6: partially vulnerable cell,
- 1: highly vulnerable cell.

Implementation advice:

- Start with a small set of hand-designed map types.
- Use clear semantic labels in code, e.g. `SAFE`, `VULNERABLE`, `HIGH_RISK`.
- Keep the matrix fixed throughout the simulation.

This is only one possible choice. You are encouraged to think about other static cell characteristics that may be relevant, especially in view of your own sub-project. Depending on the task, it may be useful to let the structure matrix encode, for example, terrain difficulty, priority level of a region, accessibility, or another fixed environmental property that is important for the corresponding analysis.

3.2 Traversability matrix

Include a matrix that determines whether a robot can move through a cell. This can be implemented either as:

- a **binary occupancy matrix** $M^{\text{occ}} \in \{0, 1\}^{n_h \times n_v}$, or
- a **debris occupancy matrix** $M^{\text{debris}}(k) \in [0, 1]^{n_h \times n_v}$.

For the first implementation, a binary representation is sufficient:

$$M_{ij}^{\text{occ}} = \begin{cases} 1, & \text{cell } (i, j) \text{ blocked,} \\ 0, & \text{cell } (i, j) \text{ free.} \end{cases}$$

Optionally, this can later be refined into a debris percentage. A simple rule can then be used such as:

A cell is traversable if its debris occupancy is below a chosen threshold.

The following matrices (scan-certainty and victim-probability) belong to the perceived state of the robot rather than to the ground-truth scenario. They are used to represent what the robot currently knows, how reliable that knowledge is, and what it believes about the possible presence of victims.

3.3 Scan-certainty matrix

Define a time-varying matrix

$$M^{\text{scan}}(k) \in [0, 1]^{n_h \times n_v},$$

where $M_{ij}^{\text{scan}}(k)$ represents the current certainty of the robot about the information associated with cell (i, j) . This matrix is part of the perceived state of the robot rather than the ground-truth environment. You can use the following update rule:

$$M_{ij}^{\text{scan}}(k) = \begin{cases} \max\{M_{ij}^{\text{scan}}(k-1) - \sigma, 0\}, & \text{if cell } (i, j) \text{ is not scanned at step } k, \\ \max\{M_{ij}^{\text{scan}}(k-1) - \sigma, \eta\}, & \text{if cell } (i, j) \text{ is scanned at step } k, \end{cases}$$

where:

- $\sigma \in [0, 1]$ is a certainty decay parameter,
- $\eta \in [0, 1]$ is the sensor accuracy.

This matrix is very useful because it introduces **partial observability** in a lightweight way: knowledge becomes stale if the robot does not revisit an area.

Recommended initialization:

$$M^{\text{scan}}(0) = \mathbf{0}^{n_h \times n_v}.$$

Recommended first simplification: At first, you may let scanning update only the current cell of the robot. Later, you can extend this to a local sensing radius.

3.4 Victim-probability matrix

Define a matrix

$$M^{\text{victim}}(k) \in [0, 1]^{n_h \times n_v},$$

where $M_{ij}^{\text{victim}}(k)$ encodes the estimated probability at time step k that a victim is present in cell (i, j) . A simple implementation of this rule is:

$$M_{ij}^{\text{victim}}(k) = \begin{cases} \frac{n_{ij}^{\text{victim,p}}(k)}{n_{\text{victim}}} M_{ij}^{\text{scan}}(k), & \text{if scanned and victims are perceived,} \\ 1 - M_{ij}^{\text{scan}}(k), & \text{if scanned and no victim is perceived,} \\ M_{ij}^{\text{victim}}(k-1), & \text{if not scanned.} \end{cases}$$

where $n_{ij}^{\text{victim,p}}(k)$ denotes the number of victims perceived in cell (i, j) at time step k , and n_{victim} denotes the assumed maximum total number of victims in the environment, used as a normalization constant. This is therefore a belief variable, not a ground-truth property of the environment. For the first implementation, an even simpler version is recommended:

- Maintain a `ground_truth_victim_map`.
- When a cell is scanned, the sensor returns a noisy detection outcome.
- The victim probability is then updated based on the scan outcome and scan certainty.

This keeps the model understandable and easy to test.

3.5 Optional fire-related matrices

If time permits, you may extend the environment with:

- a fire-state matrix $M^{\text{fire}}(k)$,
- a wind-velocity matrix $M^{\text{velocity}}(k)$,
- a wind-direction matrix $M^{\text{direction}}(k)$.

A very lightweight fire abstraction is sufficient. For example, a cell could have one of the states:

$$0 = \text{safe}, \quad 1 = \text{flammable}, \quad 2 = \text{burning}, \quad 3 = \text{burnt}.$$

3.6 Robot motion

Do **not** model velocity, acceleration, or orientation as continuous states in the common baseline. Instead:

- the robot occupies one grid cell,
- per time step it may move to one neighboring cell,
- allowed actions may be

$$\mathcal{U} = \{\text{up}, \text{down}, \text{left}, \text{right}, \text{stay}\},$$

- a move is admissible only if the target cell is inside the map and not blocked.

This is sufficient for all four sub-projects. MPC can still optimize over discrete moves or over a simplified surrogate model, and RL can learn policies in this discrete action space.

3.7 Simulator

A compact way to implement the shared simulator is the following. At each time step k , the simulator state $x(k)$ is

$$x(k) = (p(k), M^{\text{scan}}(k), M^{\text{victim}}(k), M^{\text{occ}}, M^{\text{vul}}, M^{\text{fire}}(k)),$$

where $p(k)$ is the robot position and $M^{\text{fire}}(k)$ is optional.

Given an action $u(k) \in \mathcal{U}$, the simulator performs:

1. motion update,
2. feasibility check, i.e., verify whether the target cell is inside the map and not blocked,
3. scanning update,
4. belief update,
5. optional hazard update,
6. metric and reward update.

4 Connection to the individual sub-projects

The common baseline should support the individual sub-projects, but the examples below are not meant to be exhaustive. Each of you should also think carefully about what your own sub-project will require from the shared environment and, where relevant, propose additional states, parameters, interfaces, or evaluation metrics.

4.1 Sub-project 1: Risk-aware control / robust MPC

Relevant components of the shared baseline are:

- the structure matrix as a static risk prior,
- the scan-certainty matrix as a measure of state uncertainty,
- the occupancy matrix as hard state constraints,
- the shared safety and performance metrics.

4.2 Sub-project 2: MPC-shielded reinforcement learning

This sub-project can build directly on:

- the common environment as the RL simulator,
- admissible moves and blocked cells as constraints,
- the shared step interface for agent–environment interaction,
- the shared logging and runtime measurements.

4.3 Sub-project 3: Human-aware state inference

Useful ingredients from the shared environment are:

- a small latent state describing human intent or cooperation mode,
- map regions or goal zones already defined in the common environment,
- the scan and victim maps as evidence signals,
- cost shaping or priority switching based on inferred latent states.

4.4 Sub-project 4: Coordination-aware supervisory layer

The supervisory layer can make use of:

- the region-based structure of the grid,
- uncertainty and risk maps,
- victim belief maps,
- shared task-level metrics such as coverage and discovery time.

References

- [1] F. Surma and A. Jamshidnejad, “State-Dependent Dynamic Tube MPC: A Novel Tube MPC Method with a Fuzzy Model of Disturbances,” *International Journal of Robust and Nonlinear Control*, vol. 25, no. 4, pp. 1319–1354, 2025.
- [2] O. Bastani, S. Li, and A. Xu, “Safe RL via Statistical Model Predictive Shielding,” *Robotics: Science and Systems*, pp. 1–13, 2021.
- [3] M. L. M. Patrício and A. Jamshidnejad, “Dynamic Mathematical Models of Theory of Mind for Socially Assistive Robots,” *IEEE Access*, vol. 11, pp. 103956–103975, 2023.
- [4] C. de Koning and A. Jamshidnejad, “Hierarchical Integration of Model Predictive and Fuzzy Logic Control for Combined Coverage and Target-Oriented Search-and-Rescue via Robots with Imperfect Sensors,” *Journal of Intelligent and Robotic Systems*, vol. 107, no. 40, 2023.
- [5] C. Maxwell, M. Baglioni, and A. Jamshidnejad, “Model Predictive Fuzzy Control: A Hierarchical Multi-Agent Control Architecture for Outdoor Search-and-Rescue Robots,” arXiv:2505.03257, 2025.
- [6] F. Surma and A. Jamshidnejad, “Fuzzy-logic-based model predictive control: A paradigm integrating optimal and common-sense decision making,” *Applied Soft Computing*, vol. 193, art. 114817, 2026.